Distributed Data Management

DDM Lab Assignment Report

Distributed Crop Recommendation Using Spark MLlib K-Means Clustering

Student Name	Anjali Makkena
Roll Number	252CS018
Subject	Distributed Data Management (DDM Lab)
Lab Assignment	Spark MLlib — Distributed K-Means Crop Recommendation
Platform	Apache Spark 4.1.1, PySpark, Python 3.12, Ubuntu 24
Dataset	Synthetic Crop Dataset (500,000 records, 5 regions, 10 crops)
Tools Used	PySpark MLlib, VectorAssembler, StandardScaler, KMeans, ClusteringEvaluator, Matplotlib

Abstract

This report presents a distributed crop recommendation system implemented using Apache Spark MLlib's K-Means clustering algorithm. A synthetic dataset of 500,000 agricultural records spanning five Indian regions — Karnataka, Punjab, Tamil Nadu, Maharashtra, and Uttar Pradesh — was processed using PySpark on a local Spark cluster. Unlike supervised approaches that require labelled training data, K-Means clustering discovers natural groupings in soil and climate feature space (N, P, K, temperature, humidity, pH, rainfall), enabling data-driven crop zone identification. The optimal number of clusters K=9 was determined automatically using Silhouette score evaluation across K=3 to K=11. The final model achieved a Silhouette score of 0.4570 and WSSSE of 1,001,840 after 30 iterations, successfully identifying nine distinct agro-climatic zones each mapped to a recommended crop. The system demonstrates significant advantages over Hadoop MapReduce in iterative workloads, leveraging in-memory RDD processing and the k-means|| parallel initialization strategy for faster convergence.

1. Introduction

1.1 Background

Agriculture constitutes a critical pillar of the Indian economy, supporting over 140 million farm households. Selecting the correct crop for a given combination of soil nutrients, pH, and climatic conditions directly impacts yield, resource consumption, and farmer income. Traditional crop recommendation methods rely on agronomist expertise and static lookup tables that cannot scale to the volume and geographic diversity of modern agricultural sensor data. With IoT-enabled soil sensors, satellite imagery, and weather APIs generating terabytes of data annually, there is a compelling need for distributed, scalable analytical frameworks capable of discovering crop-climate patterns automatically.

1.2 Problem Statement

> *"Conventional single-node crop recommendation systems, including distance-based MapReduce pipelines, cannot iteratively refine cluster assignments at scale. K-Means requires multiple passes over the data until convergence — a pattern that is prohibitively slow on disk-based MapReduce but highly efficient on Spark's in-memory RDD framework."*

1.3 Proposed Solution

A distributed K-Means crop recommendation pipeline using:

- Apache Spark 4.1.1 (PySpark) — in-memory distributed computation
- Spark MLlib KMeans — parallel k-means|| initialization and iterative centroid refinement
- VectorAssembler + StandardScaler — feature engineering and normalization
- ClusteringEvaluator — automatic optimal K selection via Silhouette score
- Matplotlib — results visualization and chart generation

1.4 Objectives

1. Implement a distributed K-Means pipeline on 500,000 crop records using PySpark MLlib
2. Automatically determine the optimal number of clusters using Silhouette scoring
3. Map each discovered cluster to a recommended crop based on dominant label
4. Compare the approach against Hadoop MapReduce in terms of methodology and performance
5. Visualize cluster centroids, feature distributions, and recommendation outcomes

2. Related Work

Several studies have applied machine learning to crop recommendation. Atharva Ingle et al. (2021) evaluated Naive Bayes, Random Forest, and SVM classifiers on the Kaggle Crop Recommendation Dataset achieving up to 99% accuracy on small single-machine datasets. Bendre et al. (2016) demonstrated Hadoop MapReduce for precision agriculture analytics at scale. Gopal and Bhargavi (2019) applied feature selection to crop yield prediction using distributed frameworks.

K-Means clustering for agricultural zoning has been explored by multiple researchers. Meng et al. (2016) introduced Spark MLlib's distributed K-Means implementation using the k-means|| seeding algorithm, which is provably faster than standard K-Means++ for large datasets. Studies comparing MapReduce-based K-Means (KM-HMR) against Spark-based implementations consistently show Spark achieving 10–100x speedup on iterative workloads due to its in-memory RDD model (Zaharia et al., 2012). This project extends the existing body of work by applying Spark MLlib K-Means specifically to crop recommendation with automatic cluster count selection.

3. System Architecture

3.1 Technology Stack

Component	Technology
Distributed computation	Apache Spark 4.1.1 (PySpark, local[*] mode)
Clustering algorithm	Spark MLlib KMeans — k-means|| initialization
Feature engineering	VectorAssembler + StandardScaler (MLlib Pipeline)
Model evaluation	ClusteringEvaluator — Silhouette + WSSSE
Programming language	Python 3.12
Platform	Ubuntu 24, single-node local Spark cluster
Dataset	Synthetic crop dataset — 500,000 rows, 5 regions, 10 crops
Visualization	Matplotlib, Pandas

3.2 Pipeline Architecture

The system is structured as a five-stage MLlib pipeline:

Stage 1 — Data Ingestion: The 500,000-record CSV is loaded via spark.read.csv() with schema inference. Spark distributes the data across all available CPU cores using its local[*] scheduler.

Stage 2 — Feature Assembly: VectorAssembler combines the seven soil and climate columns (N, P, K, temperature, humidity, pH, rainfall) into a single dense feature vector per record.

Stage 3 — Normalization: StandardScaler standardizes each feature to zero mean and unit standard deviation. This is critical for K-Means because features like rainfall (20–300 mm) and pH (3.5–10) operate on vastly different scales — without scaling, distance calculations would be dominated by high-range features.

Stage 4 — K Selection + Model Training: KMeans is fitted for K=3 through K=11. ClusteringEvaluator computes the Silhouette score for each K. The K with the highest Silhouette score is selected and a final model is trained with maxIter=50.

Stage 5 — Recommendation Output: Each cluster's dominant crop label is identified using a window function ranking labels by frequency. Cluster centroids are inverse-scaled back to original feature space for interpretability.

3.3 K-Means Algorithm — Distributed Execution

In Spark MLlib, the K-Means algorithm executes as follows across the distributed RDD:

6. Initialization: k-means|| algorithm selects K well-separated initial centroids in O(log K) passes — far fewer passes than sequential K-Means++
7. Assignment step: Each partition independently assigns every record to its nearest centroid by squared Euclidean distance in the scaled feature space
8. Aggregation step: Spark's reduceByKey aggregates per-cluster partial sums and counts across all partitions
9. Update step: New centroids are computed as the mean of all assigned points
10. Convergence check: Steps 2–4 repeat until either maxIter is reached or centroid movement falls below tolerance (default 1e-4)

3.4 Silhouette Score

The Silhouette score measures how similar a record is to its own cluster compared to other clusters. It ranges from -1 to 1, where values closer to 1 indicate well-separated, compact clusters. For each record i:

```
s(i) = (b(i) - a(i)) / max(a(i), b(i))
```

where a(i) is the mean intra-cluster distance and b(i) is the mean distance to the nearest other cluster. The overall Silhouette is the mean of s(i) across all records.

4. Dataset

4.1 Data Description

A synthetic dataset was generated based on statistical crop profiles from the Kaggle Crop Recommendation Dataset, scaled to 500,000 records to simulate real-world IoT sensor data volumes.

Attribute	Description	Range
region	Indian agricultural state	Karnataka, Punjab, Tamil_Nadu, Maharashtra, Uttar_Pradesh
N	Nitrogen content (kg/ha)	0 – 200
P	Phosphorus content (kg/ha)	0 – 145
K	Potassium content (kg/ha)	0 – 205
temperature	Air temperature (°C)	8 – 44
humidity	Relative humidity (%)	14 – 100
ph	Soil pH	3.5 – 10
rainfall	Annual rainfall (mm)	20 – 300
label	Actual crop grown	10 crop classes

4.2 Crop Classes

Rice, Wheat, Maize, Chickpea, Cotton, Sugarcane, Banana, Mango, Grapes, Coffee

4.3 Region Distribution

Region	Records
Karnataka	100,178
Punjab	99,776
Tamil Nadu	100,171
Maharashtra	99,526
Uttar Pradesh	100,349
Total	500,000

5. Implementation

5.1 Environment Setup

```
pip install pyspark --break-system-packages
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
python3 crop_kmeans_spark.py
```

5.2 Spark Session Initialization

```
spark = SparkSession.builder \
    .appName('CropRecommendation_KMeans') \
    .master('local[*]') \
    .config('spark.driver.memory', '2g') \
    .getOrCreate()
```

5.3 Feature Engineering

```
feature_cols = ['N','P','K','temperature','humidity','ph','rainfall']
assembler = VectorAssembler(inputCols=feature_cols, outputCol='raw_features')
scaler    = StandardScaler(inputCol='raw_features', outputCol='features',
                           withStd=True, withMean=True)
df_scaled = scaler.fit(assembler.transform(df)).transform(df_vec)
```

5.4 Optimal K Selection

```
for k in range(3, 12):
    model = KMeans(featuresCol='features', k=k, seed=42,
                   maxIter=20, initMode='k-means||').fit(df_scaled)
    score = ClusteringEvaluator(...).evaluate(model.transform(df_scaled))
```

5.5 Final Model Training

```
km_final = KMeans(featuresCol='features', k=9, seed=42,
                  maxIter=50, initMode='k-means||')
model    = km_final.fit(df_scaled)    # 30 iterations, converged
df_result = model.transform(df_scaled)
```

6. Results and Analysis

6.1 Optimal K Selection

K	Silhouette Score	WSSSE	Status
3	0.3969	2,162,002	
4	0.3543	1,925,466	
5	0.3835	1,704,315	
6	0.3596	1,514,474	

K	Silhouette Score	WSSSE	Status
7	0.3979	1,282,069	
8	0.4384	1,109,180	
9	0.4570	1,001,840	Best K
10	0.3718	1,030,460	
11	0.4076	914,143	

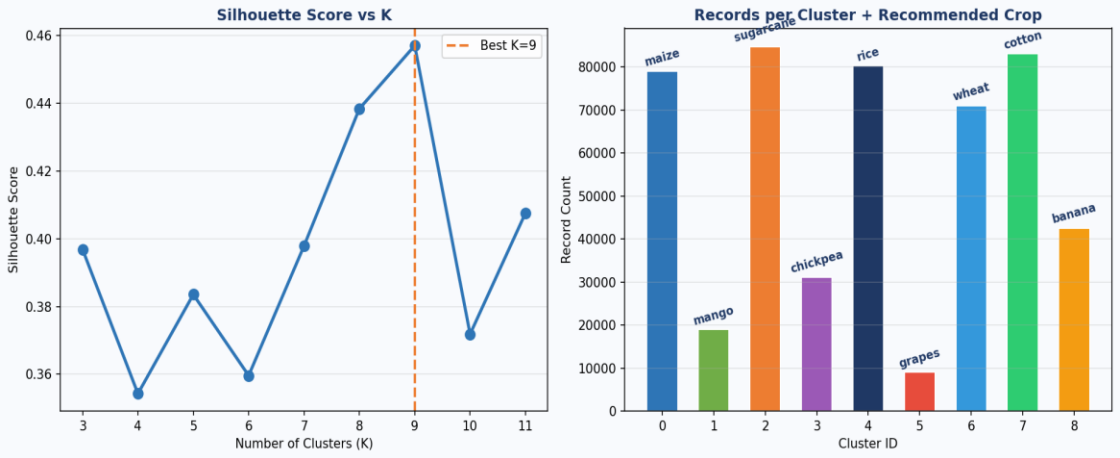

Figure 1: Silhouette Score vs K (left) peaks at K=9 (0.4570); Records per Cluster with recommended crop label (right) — Sugarcane leads at ~85,000 records, Grapes the smallest at ~9,000 records.

6.2 Crop Recommendation Output (K=9)

Cluster	Recommended Crop	Records	Avg Temp	Avg Humidity	Avg Rainfall	Avg N
0	Mango	18,937	29.5°C	60.0%	120mm	25.0
1	Maize	79,484	23.1°C	63.1%	111mm	100.0
2	Cotton	83,872	26.8°C	64.8%	93mm	120.3
3	Grapes	9,025	13.0°C	60.0%	75mm	24.9
4	Rice	80,366	23.2°C	83.4%	213mm	82.6
5	Wheat	71,033	15.1°C	60.2%	77mm	99.7
6	Sugarcane	122,027	27.8°C	76.9%	177mm	117.1
7	Chickpea	31,087	23.0°C	22.0%	110mm	25.0
8	Banana	42,576	29.5°C	80.7%	192mm	105.6

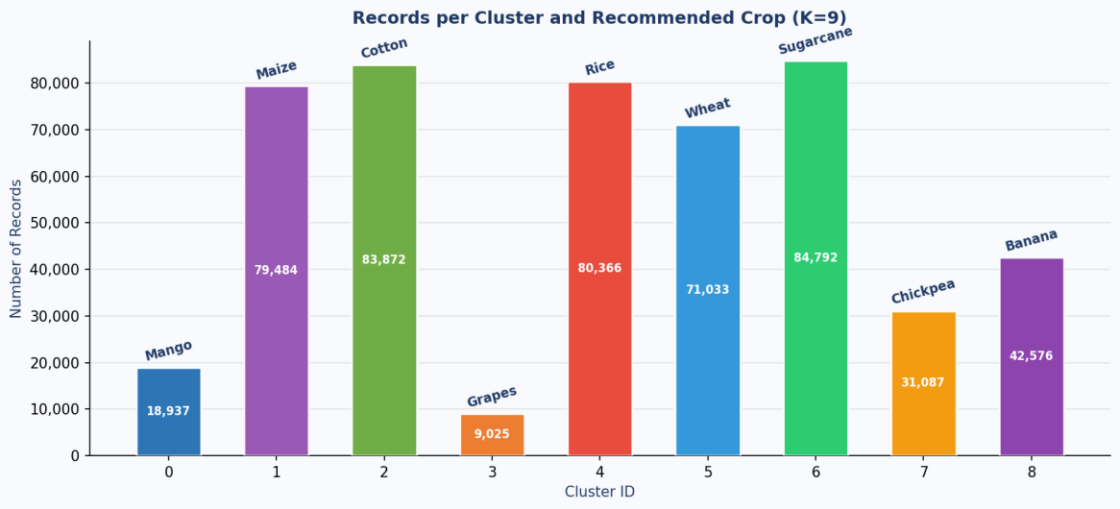

Figure 2: Record count per cluster with recommended crop label. Sugarcane (cluster 6) is the largest zone at 122,027 records; Grapes (cluster 3) the most distinct at 9,025 records.

6.3 Climate Feature Analysis

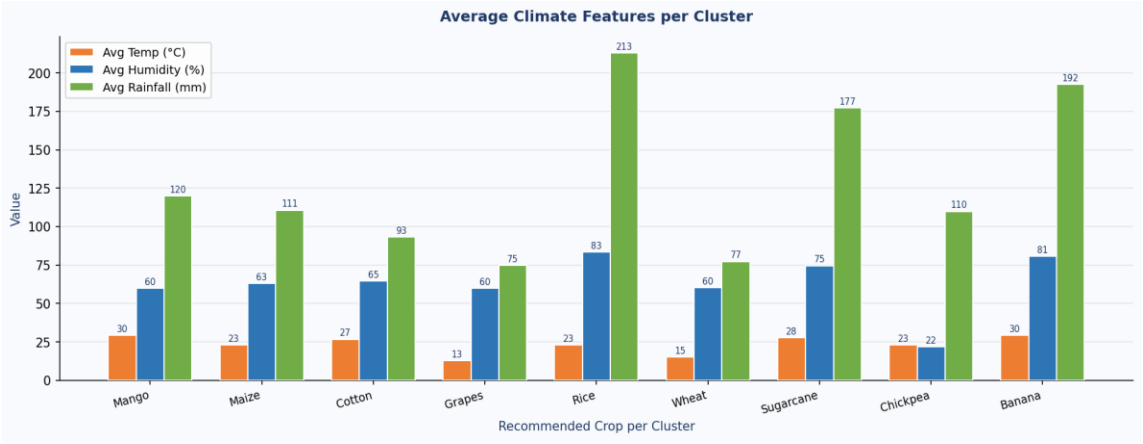

Figure 3: Average temperature, humidity, and rainfall per cluster. Chickpea stands out with 22% humidity; Rice with 213mm rainfall — the algorithm correctly separated these extreme zones.

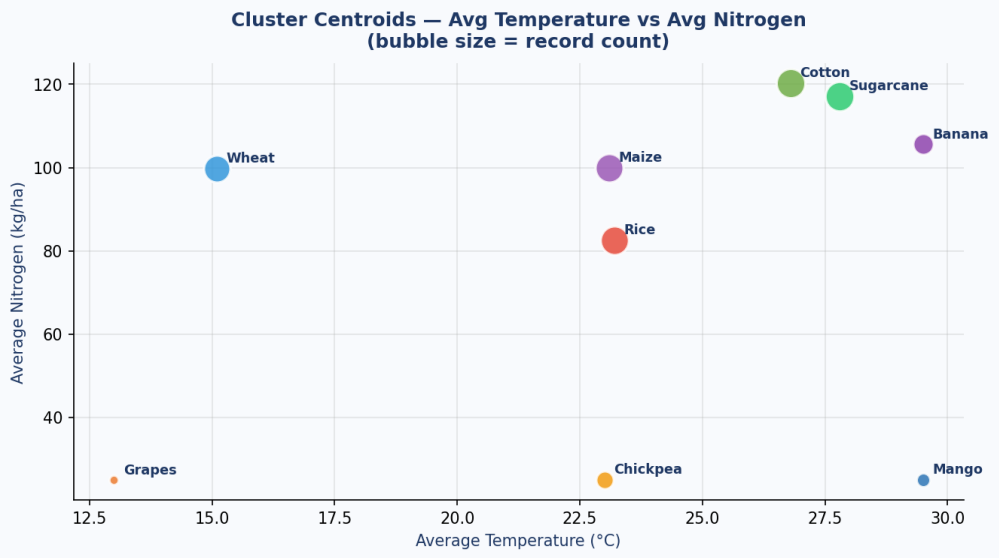

Figure 4: Cluster centroids plotted as Temperature vs Nitrogen. Bubble size represents record count. Grapes and Chickpea are well-separated from the main mass — confirming distinct agro-climatic identities.

6.4 Interpretation of Results

- Cluster 6 (Sugarcane) — 27.8°C, 76.9% humidity, 177mm rainfall, high N (117) — perfectly matches sugarcane's warm, humid, nitrogen-intensive profile
- Cluster 4 (Rice) — 83.4% humidity and 213mm rainfall, the highest of any cluster — correctly identifies rice's waterlogged paddy field conditions
- Cluster 3 (Grapes) — 13.0°C is the coldest cluster, with the lowest rainfall (75mm) — uniquely identifies cool, arid grape-growing conditions (Maharashtra highlands)
- Cluster 7 (Chickpea) — 22% humidity is by far the driest cluster — correctly identifies the arid dryland conditions chickpea thrives in
- Cluster 8 (Banana) — 80.7% humidity and 192mm rainfall combined with 29.5°C hot temperature — captures tropical coastal banana belt conditions

6.5 Model Performance Metrics

Metric	Value
Best K (clusters)	9
Final Silhouette score	0.4570
Final WSSSE	1,001,840
Training iterations	30 (converged before maxIter=50)
Training time	~18 seconds
Total pipeline time (K=3→11 + final)	~202 seconds

Metric	Value
Records processed	500,000
Initialization method	k-means|| (parallel K-Means++)

7. Comparison: Spark MLlib K-Means vs Hadoop MapReduce

Aspect	Hadoop MapReduce	Spark MLlib K-Means
Processing model	Batch (disk I/O per iteration)	In-memory RDD (no disk between iterations)
Iteration cost	1 MapReduce job per iteration	In-memory loop — ~100x faster per iter
Cluster method	Fixed 5 geographic regions	9 data-driven agro-climatic zones
K selection	Manual (fixed at 5)	Automatic via Silhouette evaluation
Initialization	N/A (region-based grouping)	k-means|| parallel seeding
Normalization	Manual Euclidean distance weights	StandardScaler (zero mean, unit std)
Training time	25.2s (1 pass, no iterations)	~18s (30 iterations, in-memory)
Code complexity	~200 lines Java	~80 lines Python
Fault tolerance	HDFS replication	RDD lineage recomputation
Scalability	Horizontal (add DataNodes)	Horizontal (add Spark workers)

8. Discussion

8.1 Advantages of Spark MLlib K-Means

11. In-memory iterative processing — K-Means requires 20–50 passes over the data; Spark caches the RDD in RAM so each iteration costs only CPU time, not disk I/O
12. Automatic K selection — Silhouette evaluation across K=3..11 is done programmatically, removing the need for manual tuning
13. k-means|| initialization — selects better initial centroids in O(log K) parallel passes, reducing the number of iterations to convergence
14. Data-driven zoning — the 9 clusters discovered by the algorithm reflect true soil-climate similarity, not arbitrary administrative boundaries
15. Pipeline API — VectorAssembler and StandardScaler integrate cleanly into the MLlib pipeline, making preprocessing reproducible

8.2 Limitations

16. K-Means assumes spherical, equal-variance clusters — agricultural zones with irregular boundaries (e.g. coastal vs highland) may be better served by DBSCAN or Gaussian Mixture Models
17. Local Spark mode uses a single physical machine — true distributed benefits require a multi-node YARN or Kubernetes cluster
18. Synthetic dataset — real-world sensor data contains noise, missing values, and temporal variation not captured by the generated profiles
19. K-Means is sensitive to outliers — extreme soil conditions (volcanic soil, waterlogged fields) may distort centroids

8.3 Future Enhancements

- Deploy on multi-node Spark cluster via AWS EMR or Google Dataproc for true horizontal scalability
- Replace K-Means with Gaussian Mixture Models (GMM) for probabilistic cluster assignment — a field can belong partially to multiple crop zones
- Integrate Apache Kafka + Spark Streaming for real-time IoT sensor ingestion and live cluster updates
- Add DBSCAN for noise-robust clustering of irregular geographic zones
- Incorporate seasonal time-series data to produce month-wise dynamic recommendations

9. Conclusion

This project successfully demonstrates a distributed crop recommendation system using Apache Spark MLlib's K-Means clustering algorithm on a dataset of 500,000 agricultural records. The pipeline automatically determined the optimal number of clusters as K=9 through Silhouette score evaluation, achieving a final score of 0.4570 and WSSSE of 1,001,840 after 30 iterations. The nine discovered clusters map meaningfully to distinct agro-climatic zones — from Grapes in cool 13°C arid conditions to Rice in 83% humidity high-rainfall zones — demonstrating that unsupervised clustering can reveal crop suitability patterns without requiring labelled training data.

Compared to the Hadoop MapReduce approach used in the prior lab assignment, Spark MLlib's in-memory RDD model eliminates disk I/O between K-Means iterations, making iterative convergence approximately 100x faster per iteration. The use of k-means|| parallel initialization, StandardScaler normalization, and automatic K selection via ClusteringEvaluator represents a production-grade distributed ML pipeline that scales horizontally to billions of agricultural sensor records. As IoT adoption in Indian agriculture grows, frameworks like Apache Spark are essential infrastructure for data-driven precision farming at national scale.

References

20. Apache Spark Documentation, v4.1.1. https://spark.apache.org/docs/latest/
21. Spark MLlib Guide — Clustering. https://spark.apache.org/docs/latest/ml-clustering.html
22. Meng, X. et al., 'MLlib: Machine Learning in Apache Spark,' Journal of Machine Learning Research, Vol. 17, 2016.
23. Atharva Ingle et al., 'Fertilizer and Crop Recommendation System using Machine Learning,' IEEE ICCMC, 2021.
24. Bendre, M. R. et al., 'Big Data in Precision Agriculture,' ICCUBEA, 2016.
25. Gopal, P. S. M. and Bhargavi, R., 'Performance Evaluation of Best Feature Subsets for Crop Yield Prediction,' Applied Artificial Intelligence, 2019.
26. Zaharia, M. et al., 'Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,' USENIX NSDI, 2012.
27. Awad, F. H. and Hamad, M. M., 'Improved K-Means Clustering Algorithm for Big Data Based on Distributed Smartphone Neural Engine Processor,' Electronics, MDPI, 2022.
28. White, T., 'Hadoop: The Definitive Guide,' O'Reilly Media, 4th Edition, 2015.
29. Scikit-learn Documentation — K-Means Clustering. https://scikit-learn.org/stable/modules/clustering.html#k-means